AI Current System Analysis

Custom AI Orchestrator Implementation

Date: February 8, 2026
Analyzed System: `ai/` folder
Purpose: Reverse-engineering documentation of the existing custom AI implementation
Status: Complete analysis - No LangChain/LangGraph concepts included

---

Table of Contents

1. System Overview
2. Main Entry Points
3. Orchestration Flow
4. RAG Pipeline
5. Tool Calling Mechanism
6. Reconciliation Workflow
7. Data Flow
8. Major Modules

---

1. System Overview

1.1 Architecture Philosophy

The `ai/` folder implements a **custom AI orchestration layer** that sits between the Angular frontend and Node.js/SQLite backend. It does NOT use any framework (no LangChain, no LangGraph). All orchestration, tool calling, and workflow management is built from scratch.

Key Design Principles:

- **Custom Tool-Calling Loop**: Manual implementation of the ReAct pattern (Reason → Act → Observe)
- **Deterministic Business Logic**: Financial operations use pure JavaScript, NOT LLMs
- **MCP Pattern**: Model Context Protocol - LLM only calls validated tool wrappers
- **Separation of Concerns**: Intent classification → Handler delegation → Tool execution

1.2 Technology Stack

```
ai/
├── Express.js Server (port 3001)
├── OpenAI API (gpt-4o-mini)
├── Custom RAG Implementation
│   ├── pdf-parse (text extraction)
│   ├── OpenAI embeddings (text-embedding-3-small)
│   ├── In-memory vector store with on-disk persistence
│   └── Cosine similarity search
├── Custom Orchestration
```

```
│    ├── Manual tool-calling loop
│    ├── Intent classification via LLM
│    └── Structured logging
└── Backend Integration
     └── Axios HTTP client to backend APIs
```

1.3 High-Level Architecture

```
┌──────────────────────────────────────────────────────────────────────┐
│                      ANGULAR FRONTEND (port 4200)                    │
│               User Interface for Expense Tracking                    │
└──────────────────────────────────────────────────────────────────────┘
                        │
                        │
                        │ HTTP (JWT Auth)
                        │
┌──────────────────────▼───────────────────────────────────────────────┐
│                  AI ORCHESTRATOR (port 3001)                         │
│                     Custom Implementation                            │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │              INTENT ROUTER (LLM-based)                       │   │
│  │  Classifies: TRANSACTIONAL | RAG_QA | RAG_COMPARE |          │   │
│  │              SYNC_RECONCILE | CLARIFICATION                  │   │
│  └──────────────┬──────────┬──────────┬──────────┬──────────────┘   │
│                 │          │          │          │                  │
│  ┌──────────────▼──┐  ┌──▼──────┐  ┌──▼──────┐  ┌──▼──────┐  ┌──▼──────────┐   │
│  │Transactional│  │RAG_QA │  │Compare│  │Sync   │  │Clarifica- │   │
│  │  Handler    │  │Handler│  │Handler│  │Handler│  │tion       │   │
│  └──────────────┘  └───────┘  └───────┘  └───────┘  └──────────────┘   │
│                 │          │          │          │                  │
│  ┌──────────────▼──────────▼──────────▼──────────▼──────────────────┐   │
│  │                    EXECUTION LAYER                               │   │
│  │  • LLM Agent (Tool Calling Loop)                                 │   │
│  │  • MCP Tools (create, list, modify, delete, clear)               │   │
│  │  • RAG Search Engine (Vector Similarity)                         │   │
│  │  • Comparison Engine (Expense Diff)                              │   │
│  │  • Reconciliation Planner (Business Rules)                       │   │
│  └──────────────────────────────────────────────────────────────────┘   │
│                                                                      │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │              DATA LAYER                                      │   │
│  │  • Vector Store (in-memory + disk persistence)               │   │
│  │  • Embeddings Cache                                          │   │
│  │  • Document Metadata                                         │   │
│  └──────────────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────────┘
                        │
                        │
                        │ HTTP (JWT Forward)
                        │
┌──────────────────────▼───────────────────────────────────────────────┐
│               NODE.JS + SQLITE BACKEND (port 3000)                   │
```

```
│          RESTful APIs for Expense CRUD, Categories, Auth          │
└──────────────────────────────────────────────────────────────────┘
```

---

2. Main Entry Points

2.1 Server Entry Point

File: `ai/server.js`

```
// Express app with production hardening
app.use(helmet());           // Security headers
app.use(cors({ ... }));      // Origin restriction
app.use(rateLimit({ ... })); // 100 req/15min limit
app.use(express.json());     // JSON body parser

// Routes
app.use('/ai', chatRoutes);   // Main chat endpoint
app.use('/ai', uploadRoutes); // PDF upload
app.use('/ai', debugRoutes);  // Debug tools (dev only)
```

Port: 3001 (hardcoded, different from backend 3000)

Production Features:

- JWT authentication middleware
- Rate limiting (prevents cost explosion)
- CORS origin validation
- Request/response logging
- Centralized error handling

2.2 Primary Chat Endpoint

File: `ai/src/routes/chat.js`

Route: `POST /ai/chat`

Request:

```
{
  "message": "Add 500 for lunch today",
  "history": [
    {"role": "user", "content": "..."},
    {"role": "assistant", "content": "..."}
  ]
}
```

Response:

```
{
  "reply": "I've added ₹500.00 for Lunch on 2026-02-08."
}
```

Processing Flow:

1. Extract JWT from `Authorization: Bearer <token>` header
2. Validate message (length, format)
3. Check for pending confirmation actions in history
4. Classify intent via `routeRequest()`
5. Route to appropriate handler
6. Return natural language response

2.3 PDF Upload Endpoint

File: `ai/src/routes/upload.js`

Route: `POST /ai/upload`

Request: `multipart/form-data` with PDF file

Processing Flow:

1. Validate PDF format (10MB limit)
2. Extract text page-by-page with `pdf-parse`
3. Clean extracted text (remove control chars)
4. Chunk text (1500 char chunks, 200 char overlap)
5. Generate embeddings via OpenAI API
6. Store in vector store with user isolation
7. Return document metadata

Response:

```
{
  "success": true,
  "message": "PDF processed successfully",
  "documentId": "doc_1738972800000_abc123",
  "chunks": 15,
  "pages": 3
}
```

---

3. Orchestration Flow

3.1 Intent Classification

File: `ai/src/router/intentRouter.js`

Method: LLM-based classification with rule-based fallback

Intent Types:

```
const INTENTS = [
  'TRANSACTIONAL',   // CRUD operations on expenses
  'RAG_QA',          // Questions about uploaded PDFs
  'RAG_COMPARE',     // Compare PDF vs app data
  'SYNC_RECONCILE',  // Sync/reconcile expenses
  'CLARIFICATION'    // Greetings, help, unclear
];
```

Classification Logic:

```
┌──────────────────────────────────────────┐
│  classifyIntent(userMessage)             │
└──────────────────────────────────────────┘
             │
             │
             ├─→ LLM Classification (primary)
             │   └─→ Few-shot prompt with examples
             │       └─→ Temperature: 0.1 (deterministic)
             │
             └─→ Rule-based Fallback (on LLM error)
                 └─→ Keyword matching
                     ├─→ "sync" → SYNC_RECONCILE
                     ├─→ "compare" → RAG_COMPARE
                     ├─→ "pdf" → RAG_QA
                     └─→ default → TRANSACTIONAL
```

Why LLM for Intent?

- Natural language understanding (handles variations)
- Context awareness (considers conversation history)
- Better than regex for ambiguous cases

Why Fallback?

- OpenAI API outage resilience
- Cost reduction in dev mode
- Faster response for obvious cases

3.2 Handler Delegation

File: `ai/src/routes/chat.js` (switch statement)

```
switch (intent) {
  case 'TRANSACTIONAL':
    reply = await handleTransactional(message, token, history, context);
```

```
      break;
    case 'RAG_QA':
      reply = await handleRagQA(message, token, userId);
      break;
    case 'RAG_COMPARE':
      reply = await handleRagCompare(message, token, userId);
      break;
    case 'SYNC_RECONCILE':
      reply = await handleSyncReconcile(message, token, userId);
      break;
    case 'CLARIFICATION':
      reply = await handleClarification(message);
      break;
  }
```

Context Propagation:

```
const context = {
  traceId,  // Request correlation ID
  userId    // From JWT token
};
```

---

4. RAG Pipeline

4.1 Document Ingestion Flow

```
┌──────────────┐
│ PDF Upload   │
│ POST /upload │
└──────────────┘
       │
       ▼
┌──────────────────────────────────────┐
│ 1. PDF Validation                    │
│    • Check mimetype                  │
│    • Verify file structure           │
│    • Enforce 10MB limit              │
└──────────────────────────────────────┘
       │
       ▼
┌──────────────────────────────────────┐
│ 2. Text Extraction (pdf-parse)       │
│    • Extract page-by-page            │
│    • Preserve page numbers for citation │
│    • Clean unicode control chars     │
│                                      │
│    Output: [                         │
│      {pageNumber: 1, text: "..."},   │
```

```
│      {pageNumber: 2, text: "..."}        │
│    ]                                     │
└──────────────────────────────────────────┘
       │
       │
       ▼
┌──────────────────────────────────────────┐
│ 3. Text Chunking (custom)                │
│    • Chunk size: 1500 chars              │
│    • Overlap: 200 chars                  │
│    • Preserves page metadata             │
│                                          │
│    Output: [                             │
│      {                                   │
│        index: 0,                         │
│        text: "...",                      │
│        pageNumber: 1,                    │
│        startChar: 0,                     │
│        endChar: 1500                     │
│      }, ...                              │
│    ]                                     │
└──────────────────────────────────────────┘
       │
       │
       ▼
┌──────────────────────────────────────────┐
│ 4. Embedding Generation (OpenAI)         │
│    • Model: text-embedding-3-small       │
│    • Dimensions: 1536                    │
│    • Batch processing (100 chunks/call)  │
│    • Cost: $0.00002 / 1K tokens          │
│                                          │
│    Output: [                             │
│      [0.123, -0.456, ...], // 1536 dims  │
│      [0.234, -0.567, ...]                │
│    ]                                     │
└──────────────────────────────────────────┘
       │
       │
       ▼
┌──────────────────────────────────────────┐
│ 5. Vector Store (in-memory + disk)       │
│    • Store chunks with embeddings        │
│    • Attach userId for isolation         │
│    • Save to data/vector-store.json      │
│                                          │
│    Structure:                            │
│    {                                     │
│      documents: [                        │
│        {                                 │
│          id: "doc_...",                  │
│          filename: "statement.pdf",      │
│          chunks: [{                      │
│            text: "...",                  │
│            embedding: [...],             │
│            pageNumber: 1,                │
│            documentId: "doc_..."         │
```

```
│          }],                              │
│          metadata: {                      │
│            userId: 123,                   │
│            storedAt: "2026-02-08T..."     │
│          }                                │
│        }                                  │
│      ]                                    │
│    }                                      │
└──────────────────────────────────────────┘
```

4.2 RAG Query Flow

File: `ai/src/handlers/ragQaHandler.js`

```
┌──────────────────────────────────────────┐
│ User Question                            │
│ "How much did I spend on groceries?"     │
└──────────────────────────────────────────┘
        │
        │
        ▼
┌──────────────────────────────────────────┐
│ 1. Similarity Search                     │
│    • Generate query embedding            │
│    • Filter by userId                    │
│    • Compute cosine similarity           │
│    • Filter by threshold (0.3)           │
│    • Sort by similarity (desc)           │
│    • Take top-k (default: 5)             │
└──────────────────────────────────────────┘
        │
        │
        ▼
┌──────────────────────────────────────────┐
│ Retrieved Chunks:                        │
│ [                                        │
│   {                                      │
│     text: "Groceries - ₹450...",         │
│     similarity: 0.87,                    │
│     filename: "statement.pdf",           │
│     pageNumber: 2                        │
│   },                                     │
│   ...                                    │
│ ]                                        │
└──────────────────────────────────────────┘
        │
        │
        ▼
┌──────────────────────────────────────────┐
│ 2. Context Building                      │
│    • Format chunks with source labels    │
│    • Concatenate for prompt              │
│                                          │
│    Context:                              │
```

```
│    [Source 1]: Groceries - ₹450...      │
│    [Source 2]: Fresh vegetables...      │
│    ...                                  │
└─────────────────────────────────────────┘
         │
         │
         ▼
┌─────────────────────────────────────────┐
│ 3. Answer Generation (LLM)              │
│    • Prompt: Context + Question         │
│    • Temperature: 0.3 (factual)         │
│    • Max tokens: 500                    │
│    • Instructions:                      │
│      - Answer from context only         │
│      - Cite sources [Source N]          │
│      - Say if answer not in docs        │
└─────────────────────────────────────────┘
         │
         │
         ▼
┌─────────────────────────────────────────┐
│ Answer with Citations                   │
│ "According to [Source 1], you spent     │
│ ₹450 on groceries. [Source 2] shows     │
│ additional vegetable purchases..."      │
└─────────────────────────────────────────┘
```

Key Implementation Details:

Cosine Similarity:

```
cosineSimilarity(vecA, vecB) {
  dotProduct = Σ(vecA[i] * vecB[i])
  normA = √Σ(vecA[i]²)
  normB = √Σ(vecB[i]²)
  return dotProduct / (normA * normB)  // Range: -1 to 1
}
```

User Isolation:

```
// Only search documents belonging to this user
getAllChunks(userId) {
  return vectorStore.documents
    .filter(doc => doc.metadata.userId === userId)
    .flatMap(doc => doc.chunks);
}
```

4.3 Expense Extraction from PDFs

File: `ai/src/rag/vectorStore.js`

Purpose: Extract structured expense data from unstructured PDF text

Method: Regex pattern matching (NOT LLM)

```
extractExpensesFromVectorStore(userId) {
  // Get all chunks for user
  const chunks = getAllChunks(userId);

  // Regular expressions for expense patterns
  const patterns = [
    // "15 Jan Groceries 450.00"
    /(\d{1,2}\s+(?:Jan|Feb|Mar|...)\s+\d{4})\s+([A-Za-z\s]+)\s+([\d,]+\.?\d*)/g,

    // "2026-01-15 | Groceries | ₹450.00"
    /(\d{4}-\d{2}-\d{2})\s*\|\s*([^\|]+)\s*\|\s*₹?([\d,]+\.?\d*)/g,

    // More patterns...
  ];

  const expenses = [];
  for (const chunk of chunks) {
    for (const pattern of patterns) {
      const matches = chunk.text.matchAll(pattern);
      for (const match of matches) {
        expenses.push({
          date: normalizeDate(match[1]),
          description: match[2].trim(),
          amount: parseFloat(match[3].replace(/,/g, '')),
          filename: chunk.filename,
          documentId: chunk.documentId,
          chunkIndex: chunk.index
        });
      }
    }
  }

  return expenses;
}
```

Why Regex Instead of LLM?

- **Deterministic**: Same input → Same output
- **Fast**: No API calls, instant results
- **Cost**: $0 (vs $0.0001-0.001 per LLM call)
- **Reliable**: No hallucinations, no API limits

Tradeoff: Regex is brittle (breaks on new formats), LLM is flexible but expensive/slow.

---

5. Tool Calling Mechanism

5.1 MCP (Model Context Protocol) Pattern

Philosophy: LLM NEVER directly calls backend APIs. All backend access goes through validated tool wrappers.

Benefits:

- **Security**: JWT authentication enforced
- **Validation**: Input sanitization before backend
- **Observability**: All tool calls logged
- **Reliability**: Timeout/retry logic
- **Type Safety**: Schema validation

5.2 Tool Definition Structure

File: `ai/src/mcp/tool.interface.js`

```
const ToolInterface = {
  definition: {
    type: "function",
    function: {
      name: "create_expense",
      description: "Adds a new expense to the tracker...",
      parameters: {
        type: "object",
        properties: {
          amount: { type: "number", description: "..." },
          category: { type: "string", description: "..." },
          description: { type: "string", description: "..." },
          expense_date: { type: "string", description: "..." }
        },
        required: ["amount", "category"]
      }
    }
  },
  run: async (args, token) => {
    // Validation
    // Normalization
    // Backend API call
    // Error handling
  }
};
```

OpenAI Function Schema: Definitions are passed directly to OpenAI's `tools` parameter.

5.3 Available Tools

File: `ai/src/mcp/tools/index.js`

```
export const tools = [
  createExpenseTool,   // Add new expense
```

```
  listExpensesTool,    // Query expenses with filters
  modifyExpenseTool,   // Update existing expense
  deleteExpenseTool,   // Remove single expense
  clearExpensesTool    // Bulk delete with filters
];
```

5.4 Tool Execution Flow

Example: User says "Add 500 for lunch today"

```
┌──────────────────────────────────────────┐
│ 1. Intent Classification                 │
│    → TRANSACTIONAL                       │
└──────────────────────────────────────────┘
        │
        │
        ▼
┌──────────────────────────────────────────┐
│ 2. Agent Loop (processChatMessage)       │
│    • Build conversation context          │
│    • Add tool definitions to prompt      │
│    • Call LLM                            │
└──────────────────────────────────────────┘
        │
        │
        ▼
┌──────────────────────────────────────────┐
│ 3. LLM Response                          │
│    {                                     │
│      tool_calls: [{                      │
│        id: "call_abc123",                │
│        function: {                       │
│          name: "create_expense",         │
│          arguments: JSON.stringify({     │
│            amount: 500,                  │
│            category: "food",             │
│            description: "lunch",         │
│            expense_date: "today"         │
│          })                              │
│        }                                 │
│      }]                                  │
│    }                                     │
└──────────────────────────────────────────┘
        │
        │
        ▼
┌──────────────────────────────────────────┐
│ 4. Tool Resolution                       │
│    • Find tool: createExpenseTool        │
│    • Parse arguments from JSON           │
│    • Validate against schema             │
└──────────────────────────────────────────┘
        │
        │
        ▼
```

```
┌──────────────────────────────────────┐
│ 5. Tool Execution                    │
│    • Validate amount (> 0)           │
│    • Normalize category (food → Food)│
│    • Parse date (today → 2026-02-08) │
│    • Fetch category_id from backend  │
│    • Call backend POST /expenses     │
│    • Handle errors                   │
└──────────────────────────────────────┘
        │
        │
        ▼
┌──────────────────────────────────────┐
│ 6. Tool Result                       │
│    {                                 │
│      success: true,                  │
│      expense: {                      │
│        id: 123,                      │
│        amount: 500,                  │
│        category_id: 1,               │
│        category_name: "Food",        │
│        description: "lunch",         │
│        date: "2026-02-08"            │
│      }                               │
│    }                                 │
└──────────────────────────────────────┘
        │
        │
        ▼
┌──────────────────────────────────────┐
│ 7. Add Tool Result to Conversation   │
│    messages.push({                   │
│      role: "tool",                   │
│      tool_call_id: "call_abc123",    │
│      content: JSON.stringify(result) │
│    })                                │
└──────────────────────────────────────┘
        │
        │
        ▼
┌──────────────────────────────────────┐
│ 8. LLM Synthesis                     │
│    • Call LLM again with tool result │
│    • LLM sees result, generates text │
│    • If more tools needed, repeat    │
│    • Else, return final response     │
└──────────────────────────────────────┘
        │
        │
        ▼
┌──────────────────────────────────────┐
│ Final Response                       │
│ "I've added ₹500.00 for Lunch on     │
│ February 8, 2026."                   │
└──────────────────────────────────────┘
```

5.5 Custom Agent Loop Implementation

File: `ai/src/llm/agent.js`

Key Features:

- Manual while loop (not framework-based)
- Iteration limit: 5 (prevents infinite loops)
- Timeout per LLM call: 60 seconds
- Supports both OpenAI function calling and text-based tool calls
- Error classification and user-friendly messages

Loop Structure:

```
async function processChatMessage(userMessage, authToken, history, context) {
  const messages = [
    { role: "system", content: getSystemPrompt() },
    ...history,
    { role: "user", content: userMessage }
  ];

  let toolIterationCount = 0;
  const MAX_ITERATIONS = 5;

  // Initial LLM call
  let response = await callLLMWithTimeout(messages);
  let responseMessage = response.choices[0].message;

  // Tool execution loop
  while (responseMessage.tool_calls && responseMessage.tool_calls.length > 0) {
    toolIterationCount++;

    if (toolIterationCount > MAX_ITERATIONS) {
      return "Error: Too many operations...";
    }

    // Add assistant's tool request to conversation
    messages.push(responseMessage);

    // Execute all tool calls
    for (const toolCall of responseMessage.tool_calls) {
      const toolName = toolCall.function.name;
      const toolArgs = JSON.parse(toolCall.function.arguments);

      // Execute tool with safety wrappers
      const result = await executeTool(toolName, toolArgs, authToken, context);

      // Add tool result to conversation
      messages.push({
        role: "tool",
        tool_call_id: toolCall.id,
        content: JSON.stringify(result)
```

```
      });
    }

    // Call LLM again with tool results
    response = await callLLMWithTimeout(messages);
    responseMessage = response.choices[0].message;
  }

  // Return final response
  return responseMessage.content;
}
```

Production Safety:

- **Iteration Limit**: Prevents cost explosion from loops
- **Timeout**: Prevents hanging requests
- **Token Usage Tracking**: Monitors costs
- **Error Classification**: Distinguishes validation vs system errors
- **Structured Logging**: Trace IDs for debugging

5.6 Tool Safety Wrappers

File: `ai/src/utils/toolExecutor.js`

```
async function executeToolSafely(toolFn, args, toolName, options) {
  const { timeout = 30000, maxRetries = 2, context = {} } = options;

  let lastError;

  for (let attempt = 0; attempt <= maxRetries; attempt++) {
    try {
      // Execute with timeout
      const result = await Promise.race([
        toolFn(args),
        new Promise((_, reject) =>
          setTimeout(() => reject(new Error('Tool timeout')), timeout)
        )
      ]);

      return result;

    } catch (error) {
      lastError = error;

      // Classify error
      const errorType = classifyError(error);

      // Don't retry validation errors
      if (errorType === 'ValidationError') {
        throw error;
      }
```

```
      // Retry transient errors
      if (errorType === 'NetworkError' && attempt < maxRetries) {
        await new Promise(resolve => setTimeout(resolve, 1000 * (attempt + 1)));
        continue;
      }

      throw error;
    }
  }
}
```

---

6. Reconciliation Workflow

6.1 Workflow Overview

Purpose: Sync expenses between PDF documents and app database

Stages:

1. **COMPARE**: Get structured diff (PDF-only, app-only, matched)
2. **PLAN**: Create deterministic reconciliation plan
3. **VALIDATE**: Pre-flight checks
4. **SYNC**: Execute via MCP tools
5. **REPORT**: Generate downloadable CSV/HTML
6. **RESPOND**: Return comprehensive summary

6.2 Stage 1: Compare

File: `ai/src/handlers/ragCompareHandler.js`

Input: User message ("compare PDF with app data")

Process:

```
// 1. Extract expenses from PDFs (regex-based)
const pdfExpenses = await extractExpensesFromVectorStore(userId);

// 2. Fetch expenses from backend
const appExpenses = await backendClient.get('/expenses', {}, authToken);

// 3. Perform code-based comparison
const diff = compareExpenses(pdfExpenses, appExpenses);

// Return structured diff
return {
  matched: [...],    // Expenses in both
  pdf_only: [...],   // Only in PDF (need to add to app)
  app_only: [...],   // Only in app (need to add to report)
  summary: {
```

```
    pdfTotal: { count, amount },
    appTotal: { count, amount }
  }
};
```

Comparison Algorithm:

File: `ai/src/comparison/expenseComparator.js`

```
function compareExpenses(pdfExpenses, appExpenses) {
  // Normalize both lists
  const normalizedPdf = pdfExpenses.map(e => ({
    amount: parseFloat(e.amount),
    date: normalizeDate(e.date),
    description: e.description.toLowerCase(),
    category: e.category.toLowerCase(),
    source: 'pdf',
    original: e
  }));

  const normalizedApp = appExpenses.map(e => ({
    amount: parseFloat(e.amount),
    date: normalizeDate(e.date),
    description: (e.description || e.category_name).toLowerCase(),
    category: e.category_name.toLowerCase(),
    source: 'app',
    original: e
  }));

  const matched = [];
  const pdfOnly = [];
  const appOnly = [];

  // Match PDF expenses to app expenses
  for (const pdfExp of normalizedPdf) {
    let bestMatch = null;
    let bestScore = 0;

    for (const appExp of normalizedApp) {
      // Skip if already matched
      if (appExp.matched) continue;

      // Amount must be close (within ₹1)
      if (Math.abs(pdfExp.amount - appExp.amount) > 1) continue;

      // Date should match
      if (pdfExp.date !== appExp.date) continue;

      // Description similarity (Jaccard)
      const similarity = jaccardSimilarity(
        pdfExp.description + ' ' + pdfExp.category,
        appExp.description + ' ' + appExp.category
```

```
      );

      if (similarity > bestScore && similarity >= 0.5) {
        bestScore = similarity;
        bestMatch = appExp;
      }
    }

    if (bestMatch) {
      bestMatch.matched = true;
      matched.push({ pdfExpense: pdfExp, appExpense: bestMatch });
    } else {
      pdfOnly.push(pdfExp.original);
    }
  }

  // Remaining app expenses
  for (const appExp of normalizedApp) {
    if (!appExp.matched) {
      appOnly.push(appExp.original);
    }
  }

  return { matched, pdfOnly, appOnly, summary: {...} };
}
```

Jaccard Similarity:

```
function jaccardSimilarity(str1, str2) {
  const tokens1 = new Set(str1.split(/\s+/));
  const tokens2 = new Set(str2.split(/\s+/));

  const intersection = new Set([...tokens1].filter(t => tokens2.has(t)));
  const union = new Set([...tokens1, ...tokens2]);

  return intersection.size / union.size;  // Range: 0 to 1
}
```

6.3 Stage 2: Plan

File: `ai/src/reconcile/reconciliationPlanner.js`

Input: Structured diff from comparison

Output: Bi-directional sync plan

```
function createReconciliationPlan(structuredDiff) {
  const plan = {
    add_to_app: [],   // PDF expenses to add to app
```

```
      add_to_pdf: [],   // App expenses to include in report
      ignored: [],      // Already matched
      rejected: [],     // Failed validation
      summary: {
        approvedForApp: 0,
        approvedForPdf: 0,
        totalMatched: 0,
        rejected: 0,
        duplicate: 0
      }
    };

    // Process PDF-only expenses (add to app)
    for (const expense of structuredDiff.pdf_only) {
      // Step 1: Validate
      const validation = validateExpense(expense);
      if (!validation.valid) {
        plan.rejected.push({ expense, reason: validation.reason });
        plan.summary.rejected++;
        continue;
      }

      // Step 2: Check for duplicates
      if (isDuplicate(expense, existingAppExpenses)) {
        plan.ignored.push({ expense, reason: 'Duplicate' });
        plan.summary.duplicate++;
        continue;
      }

      // Step 3: Normalize and approve
      const normalized = normalizeExpense(expense);
      plan.add_to_app.push({
        action: 'CREATE_EXPENSE',
        expense: normalized,
        originalExpense: expense
      });
      plan.summary.approvedForApp++;
    }

    // Process app-only expenses (add to report)
    for (const expense of structuredDiff.app_only) {
      plan.add_to_pdf.push({
        action: 'INCLUDE_IN_PDF',
        expense
      });
      plan.summary.approvedForPdf++;
    }

    // Matched expenses - no action needed
    plan.summary.totalMatched = structuredDiff.matched.length;

    return plan;
  }
```

Validation Rules:

```
const RECONCILIATION_RULES = {
  MIN_AMOUNT_THRESHOLD: 1.0,        // Skip < ₹1
  MAX_AUTO_SYNC_AMOUNT: 10000.0,    // Require manual approval > ₹10k
  ALLOW_UNDATED_EXPENSES: true,     // Allow missing dates
  ALLOW_DUPLICATE_DESCRIPTIONS: false, // Reject duplicates
  DEFAULT_CATEGORY: 'Other'         // Fallback category
};

function validateExpense(expense) {
  if (!expense.amount || expense.amount <= 0) {
    return { valid: false, reason: 'Invalid amount' };
  }

  if (expense.amount < RULES.MIN_AMOUNT_THRESHOLD) {
    return { valid: false, reason: 'Below threshold' };
  }

  if (expense.amount > RULES.MAX_AUTO_SYNC_AMOUNT) {
    return { valid: false, reason: 'Exceeds auto-sync limit' };
  }

  if (!expense.description || expense.description.trim() === '') {
    return { valid: false, reason: 'Missing description' };
  }

  return { valid: true, reason: 'Passed validation' };
}
```

Why Deterministic Planning?

- Financial reconciliation requires 100% reproducibility
- LLMs are probabilistic and can hallucinate
- Business rules must be explicit and auditable
- Compliance requires traceable decision logic

6.4 Stage 3: Validate

File: `ai/src/reconcile/syncHandler.js`

```
function validateSyncPrerequisites(plan, authToken, userId) {
  // Check authentication
  if (!authToken) {
    return { valid: false, error: 'Missing authentication token' };
  }

  // Check user ID
  if (!userId) {
```

```
    return { valid: false, error: 'Missing user ID' };
  }

  // Check if there's anything to sync
  const hasAppSync = plan.add_to_app && plan.add_to_app.length > 0;
  const hasPdfSync = plan.add_to_pdf && plan.add_to_pdf.length > 0;

  if (!hasAppSync && !hasPdfSync) {
    return { valid: false, error: 'No expenses to sync' };
  }

  return { valid: true };
}
```

6.5 Stage 4: Sync

File: `ai/src/reconcile/syncHandler.js`

Purpose: Execute plan via MCP tools (NOT direct backend calls)

```
async function executeSyncPlan(plan, authToken, userId) {
  console.log(`[Sync] Executing plan: ${plan.add_to_app.length} to app`);

  const summary = {
    totalActions: plan.add_to_app.length,
    succeeded: 0,
    failed: 0,
    skipped: 0,
    results: []
  };

  // Deduplicate (prevent double-sync on retry)
  const seen = new Set();
  const deduplicated = [];

  for (const action of plan.add_to_app) {
    const key =
`${action.expense.date}|${action.expense.amount}|${action.expense.description}`;

    if (seen.has(key)) {
      console.log(`[Sync] Skipping duplicate: ${action.expense.description}`);
      summary.skipped++;
      continue;
    }

    seen.add(key);
    deduplicated.push(action);
  }

  // Execute each action sequentially (for audit trail)
  for (const action of deduplicated) {
```

```
      try {
        // Call CreateExpenseTool (NOT direct backend API)
        const result = await executeTool(
          'create_expense',
          {
            amount: action.expense.amount,
            category: action.expense.category,
            description: action.expense.description,
            expense_date: action.expense.date
          },
          authToken,
          { userId, source: 'SYNC' }
        );

        summary.succeeded++;
        summary.results.push({
          success: true,
          expense: action.expense,
          result
        });

      } catch (error) {
        console.error(`[Sync] Failed: ${error.message}`);

        summary.failed++;
        summary.results.push({
          success: false,
          expense: action.expense,
          error: error.message
        });
      }
    }

    console.log(`[Sync] Complete: ${summary.succeeded}/${summary.totalActions}
  succeeded`);

    return summary;
  }
```

Why Sequential Execution?

- Clear audit trail (order matters)
- Easier error recovery
- Respects backend rate limits
- Simpler retry logic

Why Use MCP Tools?

- Same validation as interactive operations
- Consistent error handling
- Category lookup and normalization
- Date parsing and validation

- Backend authentication

6.6 Stage 5: Report

File: `ai/src/reports/pdfGenerator.js`

Purpose: Generate downloadable CSV + HTML reports

```
async function generateSyncedExpenseReport(authToken, userId, add_to_pdf) {
  // Fetch all app expenses
  const appExpenses = await backendClient.get('/expenses', {}, authToken);

  // Merge with add_to_pdf (app-only expenses)
  const allExpenses = [...appExpenses, ...add_to_pdf];

  // Sort by date
  allExpenses.sort((a, b) => new Date(a.date) - new Date(b.date));

  // Generate CSV
  const csv = generateCSV(allExpenses);

  // Generate HTML
  const html = generateHTML(allExpenses, {
    title: 'Synced Expense Report',
    metadata: {
      generatedAt: new Date().toISOString(),
      userId,
      totalCount: allExpenses.length,
      totalAmount: allExpenses.reduce((sum, e) => sum + e.amount, 0)
    }
  });

  // Save files
  const timestamp = Date.now();
  const csvPath = `data/reports/synced_expense_report_${userId}_${timestamp}.csv`;
  const htmlPath =
`data/reports/synced_expense_report_${userId}_${timestamp}.html`;

  await fs.writeFile(csvPath, csv);
  await fs.writeFile(htmlPath, html);

  return {
    success: true,
    files: {
      csv: { path: csvPath, size: csv.length },
      html: { path: htmlPath, size: html.length }
    },
    metadata: {...}
  };
}
```

Report Format:

- **CSV**: Standard format for Excel/Google Sheets
- **HTML**: Styled report with summary statistics

6.7 Stage 6: Respond

File: `ai/src/handlers/syncReconcileHandler.js`

Purpose: Generate comprehensive user-facing summary

```
const lines = [];
lines.push('✅ BI-DIRECTIONAL RECONCILIATION COMPLETE');
lines.push('');
lines.push('**SYNC SUMMARY (App ↔ PDF)**');
lines.push(`📤 Planned for App: ${plan.summary.approvedForApp} expenses`);
lines.push(`📤 Planned for PDF: ${plan.summary.approvedForPdf} expenses`);
lines.push(`✓ Already Matched: ${plan.summary.totalMatched} expenses`);
lines.push('');
lines.push('**EXECUTION RESULTS**');
lines.push(`✓ Succeeded: ${syncSummary.succeeded}`);
lines.push(`✗ Failed: ${syncSummary.failed}`);
lines.push(`⊘ Skipped: ${syncSummary.skipped}`);
lines.push('');
lines.push('**DOWNLOADABLE REPORT**');
lines.push(`📄 CSV: ${reportResult.files.csv.path}`);
lines.push(`📄 HTML: ${reportResult.files.html.path}`);

return lines.join('\n');
```

---

7. Data Flow

7.1 Request Flow (Transactional)

```
Frontend (Angular)
    │
    │ POST /ai/chat
    │ Headers: Authorization: Bearer <JWT>
    │ Body: { message: "Add 500 for lunch" }
    │
    ▼
AI Orchestrator (Express on port 3001)
    │
    ├─→ Auth Middleware
    │    └─→ Decode JWT → userId
    │
    ├─→ Intent Router
    │    └─→ LLM Classification → TRANSACTIONAL
    │
```

```
      ├─→ Transactional Handler
      │   └─→ Agent Loop (processChatMessage)
      │       │
      │       ├─→ LLM Call #1 (with tool definitions)
      │       │   └─→ tool_calls: [create_expense(...)]
      │       │
      │       ├─→ Tool Execution
      │       │   │
      │       │   ├─→ Validate amount
      │       │   ├─→ Normalize category
      │       │   ├─→ Parse date
      │       │   ├─→ Fetch category_id
      │       │   │
      │       │   └─→ Backend API Call
      │       │       │
      │       │       ▼
      │       │   Backend (Node + SQLite on port 3000)
      │       │       │
      │       │       ├─→ Validate JWT
      │       │       ├─→ Insert to expenses table
      │       │       └─→ Return expense object
      │       │
      │       ├─→ Add tool result to conversation
      │       │
      │       └─→ LLM Call #2 (synthesize response)
      │           └─→ "I've added ₹500.00 for Lunch..."
      │
      └─→ Return response to frontend
```

7.2 Request Flow (RAG QA)

```
Frontend (Angular)
    │
    │ POST /ai/chat
    │ Body: { message: "How much did I spend on groceries?" }
    │
    ▼
AI Orchestrator
    │
    ├─→ Intent Router → RAG_QA
    │
    ├─→ RAG QA Handler
    │   │
    │   ├─→ Generate query embedding (OpenAI API)
    │   │
    │   ├─→ Vector Store Search
    │   │   │
    │   │   ├─→ Filter by userId
    │   │   ├─→ Compute cosine similarity for each chunk
    │   │   ├─→ Filter by threshold (0.3)
    │   │   ├─→ Sort by similarity
```

```
    │   │   └─→ Return top-5 chunks
    │   │
    │   ├─→ Build context from chunks
    │   │
    │   └─→ LLM Call (with context + question)
    │       └─→ Answer with citations
    │
    └─→ Return answer to frontend
```

7.3 Request Flow (Reconciliation)

```
Frontend (Angular)
    │
    │ POST /ai/chat
    │ Body: { message: "Sync my PDF expenses" }
    │
    ▼
AI Orchestrator
    │
    ├─→ Intent Router → SYNC_RECONCILE
    │
    ├─→ Sync Reconcile Handler
    │   │
    │   ├─→ STAGE 1: Compare
    │   │   │
    │   │   ├─→ Extract PDF expenses (regex)
    │   │   ├─→ Fetch app expenses (backend API)
    │   │   └─→ Compare (code-based diff)
    │   │       └─→ Return { matched, pdf_only, app_only }
    │   │
    │   ├─→ STAGE 2: Plan
    │   │   │
    │   │   ├─→ Validate each pdf_only expense
    │   │   ├─→ Check for duplicates
    │   │   ├─→ Normalize data
    │   │   └─→ Return plan { add_to_app, add_to_pdf }
    │   │
    │   ├─→ STAGE 3: Validate
    │   │   └─→ Check prerequisites
    │   │
    │   ├─→ STAGE 4: Sync
    │   │   │
    │   │   └─→ For each expense in add_to_app:
    │   │       │
    │   │       ├─→ Call CreateExpenseTool
    │   │       │   │
    │   │       │   └─→ Backend API Call
    │   │       │       ▼
    │   │       │   Backend (insert expense)
    │   │       │
    │   │       └─→ Track success/failure
```

```
    │   │
    │   ├─→ STAGE 5: Report
    │   │   │
    │   │   ├─→ Fetch all app expenses
    │   │   ├─→ Merge with add_to_pdf
    │   │   ├─→ Generate CSV + HTML
    │   │   └─→ Save to data/reports/
    │   │
    │   └─→ STAGE 6: Respond
    │       └─→ Return comprehensive summary
    │
    └─→ Return summary to frontend
```

7.4 Data Persistence

```
AI Orchestrator File System:

ai/
├── data/
│   ├── vector-store.json         ← Vector store persistence
│   │   {
│   │     documents: [
│   │       {
│   │         id: "doc_...",
│   │         filename: "statement.pdf",
│   │         chunks: [
│   │           {
│   │             index: 0,
│   │             text: "...",
│   │             embedding: [0.123, ...],  ← 1536 dimensions
│   │             pageNumber: 1,
│   │             documentId: "doc_..."
│   │           }
│   │         ],
│   │         metadata: {
│   │           userId: 123,
│   │           storedAt: "2026-02-08T..."
│   │         }
│   │       }
│   │     ]
│   │   }
│   │
│   └── reports/                    ← Reconciliation reports
│       ├── synced_expense_report_123_1738972800000.csv
│       └── synced_expense_report_123_1738972800000.html

Backend Database (SQLite):

backend/
└── database.sqlite
    ├── users              ← User accounts
```

```
    ├── categories         ← Expense categories
    └── expenses           ← Expense transactions
        {
          id: 123,
          user_id: 456,
          category_id: 1,
          amount: 500.00,
          description: "lunch",
          date: "2026-02-08",
          created_at: "2026-02-08T10:30:00Z"
        }
```

---

8. Major Modules

8.1 Module Summary

Module	File Path	Purpose	Technology
Server	`server.js`	Express app with security	Express, Helmet, CORS, Rate Limiting
Intent Router	`src/router/intentRouter.js`	LLM-based intent classification	OpenAI API
Handlers	`src/handlers/*.js`	Intent-specific orchestration	Custom logic
LLM Agent	`src/llm/agent.js`	Custom tool-calling loop	OpenAI function calling
System Prompt	`src/llm/systemPrompt.js`	Agent instructions	Prompt engineering
MCP Tools	`src/mcp/tools/*.js`	Backend API wrappers	Tool pattern
Tool Executor	`src/utils/toolExecutor.js`	Safety wrappers (timeout, retry)	Production hardening
RAG: Vector Store	`src/rag/vectorStore.js`	In-memory vectors with persistence	JSON file
RAG: Embeddings	`src/rag/embeddings.js`	OpenAI embedding API	text-embedding-3-small
RAG: Chunking	`src/rag/chunker.js`	Text chunking (1500 chars, 200 overlap)	Pure JavaScript
RAG: Search	`src/rag/search.js`	Cosine similarity search	Pure JavaScript

Module	File Path	Purpose	Technology
Comparison	`src/comparison/expenseComparator.js`	Expense diff engine	Jaccard similarity
Reconciliation	`src/reconcile/*.js`	Sync planning and execution	Deterministic logic
Reports	`src/reports/pdfGenerator.js`	CSV/HTML report generation	fs, Handlebars
Validators	`src/validators/*.js`	Input validation	Pure JavaScript
Utils	`src/utils/*.js`	Shared utilities	Various

8.2 Module: Intent Router

Purpose: Classify user intent and route to handler

Classification Method: LLM with fallback

Prompt Engineering:

```
const prompt = `
You are an intent classifier for an expense tracker AI system.
Classify the user's message into ONE of these intents:

1. TRANSACTIONAL - User wants to add, modify, delete, or list expenses
2. RAG_QA - User asks questions about their uploaded PDF documents
3. RAG_COMPARE - User wants to compare PDF data with app data
4. SYNC_RECONCILE - User wants to sync PDF expenses into the app
5. CLARIFICATION - Ambiguous, greeting, or out-of-scope

User message: "${userMessage}"

Respond with ONLY the intent name. No explanation.
`;
```

Temperature: 0.1 (deterministic)

Fallback: Keyword matching if LLM fails

8.3 Module: LLM Agent

Purpose: Execute tool-calling loop for transactional operations

Key Features:

- Manual while loop (not framework)
- Iteration limit: 5
- Timeout: 60s per LLM call
- Supports OpenAI function calling API
- Fallback text parsing for non-function-calling models

System Prompt Structure:

```
function getSystemPrompt(supportsNativeTools) {
  const today = new Date().toISOString().split('T')[0];

  return `You are an AI assistant for an expense tracking application.

Current date: ${today}

Your capabilities:
1. Create expense records
2. List expenses with filters
3. Modify existing expenses
4. Delete expenses
5. Clear multiple expenses

When users mention adding/spending money:
- Call create_expense tool
- ALWAYS include amount as a number
- Use category names like "food", "transport", "entertainment"
${supportsNativeTools ?
  '- Use the function calling API' :
  '- Format tool calls as: <|python_start|>tool_name(arg="value")<|python_end|>'
}

...additional instructions...
`;
}
```

8.4 Module: MCP Tools

Purpose: Safe wrappers around backend APIs

Common Pattern:

```
export const createExpenseTool = {
  // OpenAI function definition
  definition: {
    type: "function",
    function: {
      name: "create_expense",
      description: "...",
      parameters: { /* JSON Schema */ }
    }
  },

  // Implementation
  run: async (args, token) => {
    // Step 1: Validate
    const validatedAmount = validateAmount(args.amount);
```

```
    // Step 2: Normalize
    const normalizedCategory = normalizeCategory(args.category);
    const parsedDate = parseDate(args.expense_date || 'today');

    // Step 3: Fetch category ID
    const category = await findCategoryByName(normalizedCategory, token);

    // Step 4: Call backend
    const result = await backendClient.post('/expenses', {
      amount: validatedAmount,
      category_id: category.id,
      description: args.description,
      date: parsedDate
    }, token);

    return result;
  }
};
```

Benefits:

- Input validation before backend
- Consistent error handling
- Category name → ID lookup
- Date parsing and normalization
- JWT authentication

8.5 Module: RAG Vector Store

Purpose: Store and search document embeddings

Data Structure:

```
{
  documents: [
    {
      id: "doc_1738972800000_abc123",
      filename: "statement.pdf",
      chunks: [
        {
          index: 0,
          text: "Transaction history...",
          embedding: [0.123, -0.456, ...], // 1536 dims
          pageNumber: 1,
          startChar: 0,
          endChar: 1500,
          length: 1500,
          documentId: "doc_1738972800000_abc123",
          filename: "statement.pdf"
        }
```

```
      ],
      metadata: {
        userId: 123,
        storedAt: "2026-02-08T10:30:00Z",
        uploadedBy: "user@example.com"
      }
    }
  ],
  metadata: {
    totalDocuments: 1,
    totalChunks: 15,
    lastUpdated: "2026-02-08T10:30:00Z"
  }
}
```

Operations:

- `storeDocument(document)`: Add new document
- `getAllChunks(userId)`: Get chunks for user
- `deleteDocument(documentId, userId)`: Remove document
- `getUserDocuments(userId)`: List user's documents
- `saveVectorStore()`: Persist to disk
- `loadVectorStore()`: Load from disk

User Isolation: All operations filter by `userId`

8.6 Module: RAG Search Engine

Purpose: Find relevant chunks via similarity search

Algorithm:

```
async function searchSimilarChunks(queryText, userId, topK = 5, options = {}) {
  // 1. Get user's chunks
  const chunks = getAllChunks(userId);

  // 2. Embed query
  const queryEmbedding = await generateEmbedding(queryText);

  // 3. Compute similarity for all chunks
  const results = chunks.map(chunk => ({
    text: chunk.text,
    similarity: cosineSimilarity(queryEmbedding, chunk.embedding),
    filename: chunk.filename,
    pageNumber: chunk.pageNumber,
    metadata: { ... }
  }));

  // 4. Filter by threshold
  const filtered = results.filter(r => r.similarity >= 0.3);
```

```
  // 5. Sort and take top-k
  return filtered
    .sort((a, b) => b.similarity - a.similarity)
    .slice(0, topK);
}
```

Cosine Similarity:

```
similarity(A, B) = (A · B) / (||A|| × ||B||)

Range: -1 to 1
- 1.0 = identical
- 0.0 = orthogonal (unrelated)
- -1.0 = opposite
```

8.7 Module: Expense Comparator

Purpose: Deterministic diff between PDF and app expenses

Matching Logic:

1. Amount must be within ₹1
2. Date must match (if both have dates)
3. Description similarity >= 0.5 (Jaccard)

Output:

```
{
  matched: [
    { pdfExpense: {...}, appExpense: {...}, confidence: 0.87 }
  ],
  pdfOnly: [
    { amount: 500, date: "2026-02-08", description: "lunch" }
  ],
  appOnly: [
    { amount: 300, date: "2026-02-07", category_name: "Transport" }
  ],
  summary: {
    pdfTotal: { count: 10, amount: 5000 },
    appTotal: { count: 8, amount: 4200 }
  }
}
```

8.8 Module: Reconciliation Planner

Purpose: Create deterministic sync plans

Validation Rules:

- Amount > ₹1 (MIN_AMOUNT_THRESHOLD)
- Amount < ₹10,000 (MAX_AUTO_SYNC_AMOUNT)
- Description required
- Duplicate detection

Plan Structure:

```
{
  add_to_app: [
    {
      action: 'CREATE_EXPENSE',
      expense: {
        amount: 500,
        category: 'Food',
        description: 'lunch',
        date: '2026-02-08'
      },
      originalExpense: { /* PDF data */ }
    }
  ],
  add_to_pdf: [
    {
      action: 'INCLUDE_IN_PDF',
      expense: { /* App data */ }
    }
  ],
  ignored: [
    { expense: {...}, reason: 'Duplicate' }
  ],
  rejected: [
    { expense: {...}, reason: 'Amount below threshold' }
  ],
  summary: {
    approvedForApp: 3,
    approvedForPdf: 2,
    totalMatched: 5,
    rejected: 1,
    duplicate: 1
  }
}
```

8.9 Module: Production Utilities

Cost Tracking (`src/utils/costTracking.js`):

```
function recordUsage(model, tokens, traceId, userId) {
  const cost = calculateCost(model, tokens);
  console.log(`[Cost] ${traceId} | ${userId} | ${model} | ${tokens} tokens |
$${cost}`);
```

```
    // Could save to database for billing
  }
```

Error Classification (`src/utils/errorClassification.js`):

```
function classifyError(error) {
  // Validation errors (user's fault)
  if (error.message.includes('Invalid') || error.message.includes('required')) {
    return { type: 'ValidationError', userMessage: error.message };
  }

  // Network errors (retry possible)
  if (error.code === 'ECONNREFUSED' || error.code === 'ETIMEDOUT') {
    return { type: 'NetworkError', userMessage: 'Service temporarily unavailable'
};
  }

  // System errors (don't expose details)
  return { type: 'SystemError', userMessage: 'An error occurred. Please try
again.' };
}
```

Structured Logging (`src/utils/logger.js`):

```
function createLogger(module) {
  return {
    info: (message, meta) => console.log(JSON.stringify({
      level: 'INFO',
      module,
      message,
      ...meta,
      timestamp: new Date().toISOString()
    })),
    error: (message, meta) => console.error(JSON.stringify({
      level: 'ERROR',
      module,
      message,
      ...meta,
      timestamp: new Date().toISOString(),
      stack: meta.error?.stack
    }))
  };
}
```

Appendix A: File Structure

```
ai/
├── server.js                    ← Main entry point
├── package.json                 ← Dependencies
├── .env                         ← Environment config
│
├── src/
│   ├── routes/
│   │   ├── chat.js              ← POST /ai/chat (main endpoint)
│   │   ├── upload.js            ← POST /ai/upload (PDF ingestion)
│   │   └── debug.js             ← Debug utilities
│   │
│   ├── router/
│   │   └── intentRouter.js      ← Intent classification
│   │
│   ├── handlers/
│   │   ├── transactionalHandler.js      ← CRUD operations
│   │   ├── ragQaHandler.js              ← Document Q&A
│   │   ├── ragCompareHandler.js         ← Expense comparison
│   │   ├── syncReconcileHandler.js      ← Reconciliation orchestrator
│   │   └── clarificationHandler.js      ← Help/greetings
│   │
│   ├── llm/
│   │   ├── agent.js             ← Custom tool-calling loop
│   │   └── systemPrompt.js      ← Agent instructions
│   │
│   ├── mcp/
│   │   ├── tool.interface.js    ← Tool pattern reference
│   │   └── tools/
│   │       ├── index.js         ← Tool registry
│   │       ├── createExpense.js ← Add expense
│   │       ├── listExpenses.js  ← Query expenses
│   │       ├── modifyExpense.js ← Update expense
│   │       ├── deleteExpense.js ← Remove expense
│   │       └── clearExpenses.js ← Bulk delete
│   │
│   ├── rag/
│   │   ├── vectorStore.js       ← In-memory + disk persistence
│   │   ├── embeddings.js        ← OpenAI embedding API
│   │   ├── chunker.js           ← Text chunking
│   │   └── search.js            ← Similarity search
│   │
│   ├── comparison/
│   │   └── expenseComparator.js ← Expense diff engine
│   │
│   ├── reconcile/
│   │   ├── reconciliationPlanner.js  ← Sync planning (deterministic)
│   │   └── syncHandler.js            ← Plan execution
│   │
│   ├── reports/
│   │   └── pdfGenerator.js      ← CSV/HTML report generation
│   │
│   ├── validators/
│   │   └── expenseValidator.js  ← Input validation
```

```
│   │
│   ├── utils/
│   │   ├── backendClient.js      ← Axios wrapper
│   │   ├── categoryCache.js      ← Category lookup
│   │   ├── dateNormalizer.js     ← Date parsing
│   │   ├── pdfExtractor.js       ← pdf-parse wrapper
│   │   ├── toolExecutor.js       ← Tool safety wrappers
│   │   ├── logger.js             ← Structured logging
│   │   ├── costTracking.js       ← Token/cost monitoring
│   │   └── errorClassification.js ← Error handling
│   │
│   └── middleware/
│       ├── auth.js               ← JWT authentication
│       └── errorHandler.js       ← Centralized error handler
│
├── data/
│   ├── vector-store.json         ← Vector store persistence
│   └── reports/                  ← Generated CSV/HTML files
│
└── tests/
    ├── chunking.test.js
    ├── comparison.test.js
    ├── search.test.js
    └── validator.test.js
```

Appendix B: Key Dependencies

Production Dependencies:

```
{
  "express": "^4.18.0",           // Web server
  "cors": "^2.8.5",               // CORS handling
  "helmet": "^7.1.0",             // Security headers
  "express-rate-limit": "^7.1.5", // Rate limiting
  "dotenv": "^16.0.0",            // Environment config
  "openai": "^4.20.0",            // OpenAI API client
  "axios": "^1.6.0",              // HTTP client
  "multer": "^1.4.5-lts.1",       // File upload
  "pdf-parse": "^1.1.1",          // PDF text extraction
  "jsonwebtoken": "^9.0.2"        // JWT handling
}
```

Environment Variables:

```
# OpenAI
LLM_API_KEY=sk-...
LLM_MODEL=gpt-4o-mini
LLM_BASE_URL=https://api.openai.com/v1  # Optional
```

```
# Backend
BACKEND_BASE_URL=http://localhost:3000
JWT_SECRET=your-secret-key

# Security
ALLOWED_ORIGINS=http://localhost:4200
RATE_LIMIT_WINDOW_MS=900000  # 15 minutes
RATE_LIMIT_MAX_REQUESTS=100

# Production
NODE_ENV=production  # or development
PORT=3001
ENABLE_DEBUG_ROUTES=false
```

Appendix C: API Endpoints

AI Orchestrator (port 3001)

Chat:

```
POST /ai/chat
Headers:
  Authorization: Bearer <JWT>
  Content-Type: application/json
Body:
  {
    "message": "Add 500 for lunch today",
    "history": [...]  // Optional
  }
Response:
  {
    "reply": "I've added ₹500.00 for Lunch..."
  }
```

Upload:

```
POST /ai/upload
Headers:
  Authorization: Bearer <JWT>
  Content-Type: multipart/form-data
Body:
  file: <PDF file>
Response:
  {
    "success": true,
    "documentId": "doc_...",
    "chunks": 15,
```

```
    "pages": 3
  }
```

Health Check:

```
GET /health
Response:
  {
    "status": "OK",
    "service": "AI Orchestrator"
  }
```

Backend (port 3000)

Expenses:

```
GET /expenses?limit=50&offset=0&category_id=1
POST /expenses
  Body: { amount, category_id, description, date }
PUT /expenses/:id
  Body: { amount, category_id, description, date }
DELETE /expenses/:id
```

Categories:

```
GET /categories
Response:
  [
    { id: 1, name: "Food", user_id: null },
    { id: 2, name: "Transport", user_id: null },
    ...
  ]
```

Appendix D: Production Considerations

Security

- [x] JWT authentication on all routes
- [x] Helmet security headers
- [x] CORS origin validation
- [x] Rate limiting (100 req/15min)
- [x] Input validation before backend
- [x] SQL injection protection (parameterized queries in backend)
- [x] File upload size limits (10MB)

Observability

- [x] Structured logging (JSON)
- [x] Request trace IDs
- [x] Token usage tracking
- [x] Error classification
- [x] Audit trail for reconciliation

Reliability

- [x] Timeout protection (60s LLM, 30s tools)
- [x] Retry logic for transient failures
- [x] Iteration limits (prevents infinite loops)
- [x] Graceful error handling
- [x] Fallback classification (keyword-based)

Performance

- [x] Vector search in-memory (fast)
- [x] Embedding batch processing
- [x] Category caching
- [x] Connection pooling (backend)

Scalability Limitations

⚠ In-memory vector store (doesn't scale horizontally)
⚠ No distributed tracing (single-instance only)
⚠ File-based persistence (not cloud-native)

Conclusion

The `ai/` folder implements a **production-ready custom AI orchestration layer** without using any framework. Key architectural decisions:

1. **Custom Tool-Calling Loop**: Manual implementation provides full control and observability
2. **Deterministic Financial Logic**: LLMs only for NL interfaces, not decisions
3. **MCP Pattern**: Validated tool wrappers prevent direct backend access
4. **RAG via Regex**: Fast, deterministic expense extraction
5. **Bi-directional Reconciliation**: Additive-only sync (never auto-delete)
6. **User Isolation**: All data scoped by userId
7. **Production Hardening**: Rate limiting, timeouts, retries, logging

The system successfully bridges natural language understanding (LLM) with structured data operations (backend APIs) while maintaining security, reliability, and auditability required for financial applications.

End of Analysis